

Assignment 4 — How Few Components Can You Get Away With?

Feature Engineering & MLOps · Dimensionality Reduction (PCA Practical)

Course	Feature Engineering & MLOps
Assignment No.	4
Topic	PCA — Explained Variance, Scree Plot, Choosing the Optimal Number of Components
Marks	____ / 100
Due Date	_____

1. Objective

You are given a dataset with **60 numeric features**. Your job is to find the SMALLEST number of principal components that lets a Linear Regression model achieve roughly the SAME test accuracy as using all 60 raw features — and to explain, using a scree plot and explained variance, exactly why that number is what it is.

This is not a hypothetical exercise: real datasets are often full of redundant, correlated columns. Finding the true "effective dimensionality" of your data is one of the most practically useful skills PCA gives you — smaller models, faster training, and often less overfitting, all without sacrificing accuracy.

2. Learning Outcomes

- Compute and interpret explained variance ratio and cumulative explained variance.
- Build and correctly read a scree plot to identify an "elbow."
- Compare model accuracy across a range of component counts, using a held-out test set.
- Identify the smallest number of components that matches (or nearly matches) full-feature accuracy, and justify that choice with evidence, not a guess.
- Explain why PCA can sometimes make a model MORE accurate, not just smaller.

3. Dataset

Use `data/raw/pca_regression_data.csv` — 800 rows, 60 numeric features (`feature_1 ... feature_60`), and a continuous target column `score`.

NOTE: You are NOT told in advance how many features are truly independent — that is exactly what this assignment asks you to discover empirically, using the tools from the PCA Practical session.

4. Tasks

4.1 — Baseline: all 60 raw features

1. Load the dataset. Split into train (75%) and test (25%) using a fixed `random_state`.
2. Standardize the 60 features (fit the scaler on train only).
3. Train a `LinearRegression` model on all 60 standardized features. Report the test R^2 .

4.2 — PCA: explained variance and the scree plot

1. Fit a PCA (no `n_components` limit) on the standardized TRAINING features.
2. Plot the scree plot: explained variance ratio (y-axis) against component number (x-axis), for at least the first 20 components.
3. Plot cumulative explained variance against component number, on a separate chart, with a horizontal reference line at 95%.
4. From the scree plot alone, identify where the "elbow" appears to be — write down your visual estimate BEFORE doing anything else.

4.3 — Find the smallest component count that matches baseline accuracy

1. For component counts `[2, 4, 6, 8, 10, 12, 15, 20, 30, 40]`: transform the training and test features with a PCA fit to that many components, train a fresh `LinearRegression` on the transformed training data, and record the test R^2 .
2. Plot test R^2 against number of components on one chart, with a horizontal reference line at the Section 4.1 baseline R^2 .
3. Identify the SMALLEST number of components whose R^2 is within 0.01 of the baseline R^2 (or exceeds it). This is your answer to "how few components can you get away with?"
4. Compare this number to your visual scree-plot elbow estimate from Section 4.2, Step 4 — did they agree?

4.4 — Explain it

In 4-6 sentences, using your OWN numbers from Sections 4.1-4.3: explain why the model needs so few components to match (or beat) the accuracy of all 60 raw features. What does this tell you about how the original 60 features were related to each other? (Hint: think back to the "true latent factors" idea from the PCA Practical session — you don't need to know the exact number of true factors used to generate this dataset, but your answer should reflect the same underlying reasoning.)

5. Submission Instructions

1. In your existing course GitHub repository, use (or create) the top-level `Assignments` folder.
2. Add your completed notebook as `Assignment4_PCA_Components.ipynb`. Run every cell top-to-bottom with no errors, and keep outputs (tables, all 3 charts) visible before committing.
3. Structure the notebook with clear markdown headers matching Sections 4.1-4.4 above.
4. Commit and push your changes:

```
git add Assignments/Assignment4_PCA_Components.ipynb
git commit -m "Assignment 4: PCA component selection"
git push
```

Finally, share your GitHub repository link with the instructor by the due date above. Grading is done directly from the `Assignments` folder — make sure `Assignments 1` through `4` are all there and pushed before the deadline.

IMPORTANT: Only the notebook committed to your `Assignments` folder on GitHub will be evaluated. Local-only or emailed notebooks will not be graded.

6. Grading Rubric

Criteria	Weight
Baseline (all 60 features) correctly trained and evaluated	15%
Scree plot and cumulative explained variance plot, correctly built and labeled	20%
R^2 vs. number of components chart, covering all required component counts	20%
Correct identification of the smallest component count matching baseline accuracy	20%
Written explanation (Section 4.4) shows genuine understanding, not just a number	15%
Code quality: clear structure, markdown headers, readable variable names	10%

7. Bonus (Optional, +10% extra credit)

Repeat Section 4.3 using a `RandomForestRegressor` instead of `Linear Regression`. Does the SAME number of components still match baseline accuracy for a tree-based model? If the answer differs, explain why, referencing how tree-based models use features differently than linear models (revisit the `Categorical Encoding` session's discussion of linear vs. tree-based model sensitivity if you need a refresher).